

Exercise 2: Protocol Exploration & Local REST APIs

Course: Architectural Convergence in Computer Engineering: IoT, Middleware, Cloud, and Database Systems

Week: 2 (Hands-On Laboratory Exercise)

Objective: To deploy a local Node-RED middleware container, architect a RESTful HTTP server with distinct methods (GET/POST/PATCH/DELETE), monitor raw data via debugging consoles, and validate endpoint connectivity using Postman.

Step 1: Run the Node-RED Docker Image on the Local Network

To act as our local API server, we must spin up the Node-RED container we downloaded in Week 1 and expose its internal port to your computer's Local Area Network (LAN) so external devices (like Postman or an ESP32) can reach it.

1. **Open Docker Desktop:** Ensure the Docker daemon is running in the background.
2. **Launch via Docker Desktop:** Open your computer's Docker Desktop click the run button for the NodeRed image, and use the settings as follow:
 - **Ports:** The -p 1880:1880 flag is critical. It binds port 1880 inside the isolated container to port 1880 on your physical machine's LAN IP address.
 - **Volumes:** If you want to store your data in host rather than inside the docker itself.



Run a new container

nodered/node-red:latest

Optional settings

Container name

NodeRed

A random name is generated if you do not provide one.

Ports

Enter "0" to assign randomly generated host ports.

Host port

1880

:1880/tcp

Volumes

Host path

~/yewguan/Desktop/NodeRed

Container path

/data

Environment variables

Variable

Value

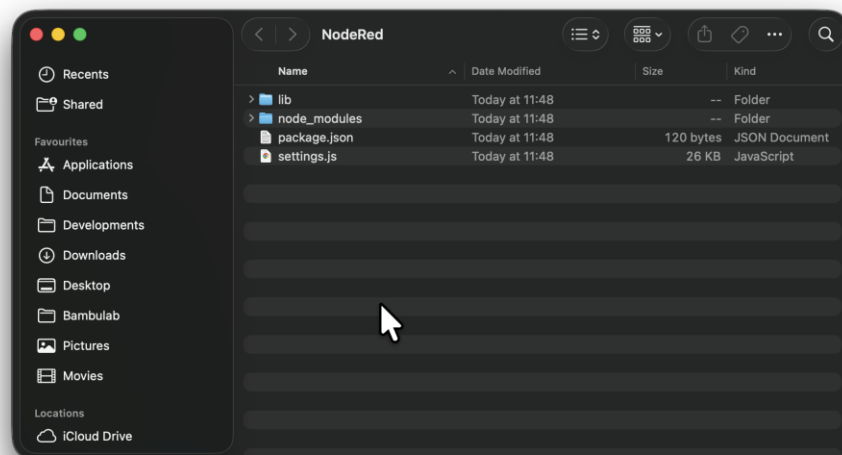
Cancel

Run

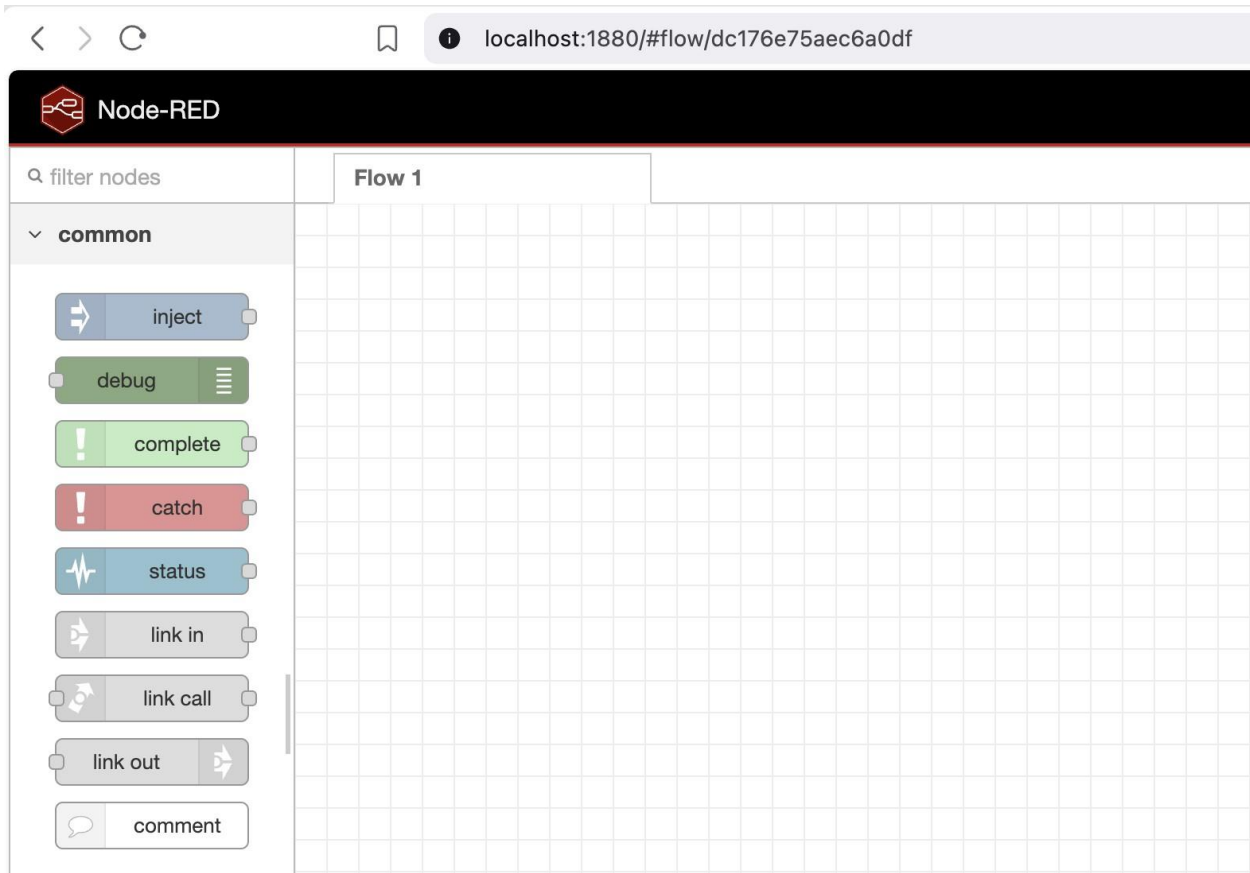
○

3. Verify Execution:

- Check data created in the folder of the host path that you configured



- Open your web browser and navigate to <http://localhost:1880>. The Node-RED visual workspace should load.

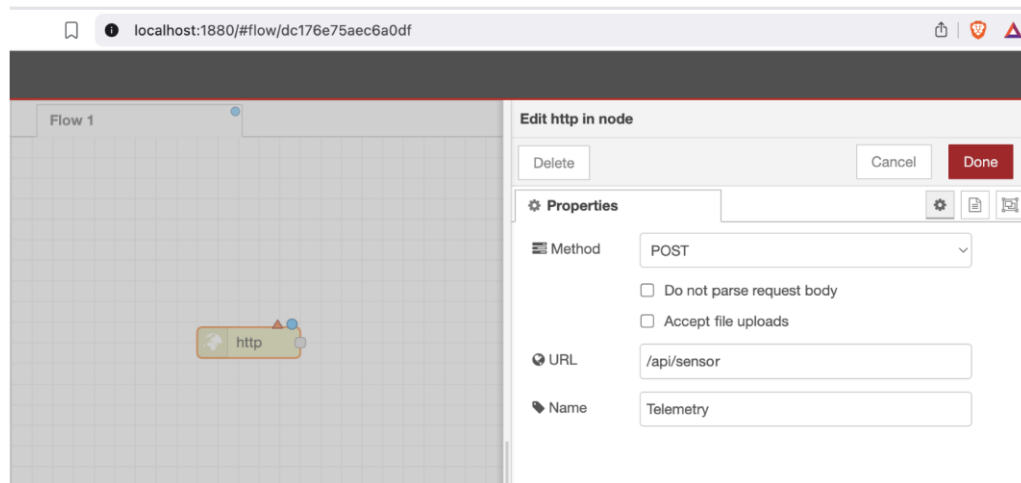


Step 2: Design Node-RED for HTTP Requests and Responses

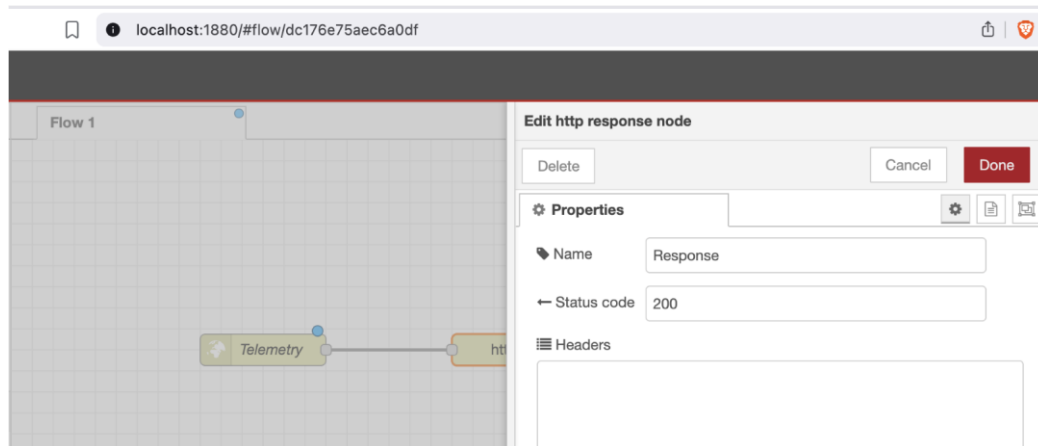
Now we will architect a RESTful API endpoint that can receive incoming sensor telemetry via an HTTP POST request and acknowledge it.

1. **Add an HTTP Input Node:** Drag an `http in` node from the left palette onto the workspace. Double-click it to configure:

- **Method:** POST
- **URL:** /api/sensor
- **Name:** Receive Telemetry



2. **Add an HTTP Response Node:** Drag an [http response](#) node onto the workspace. This node is required to close the HTTP transaction by sending a status code back to the client. Double-click it:
 - **Status Code:** 200 (which signifies "OK").



3. **Wire the Nodes:** Click and drag a wire connecting the output port of the http in node directly to the input port of the http response node.
4. **Deploy:** Click the red **Deploy** button in the top right corner to make your API go live.

Step 3: Test the Endpoint using Postman

With our server running, we will act as the client using Postman to simulate an IoT device sending telemetry data.

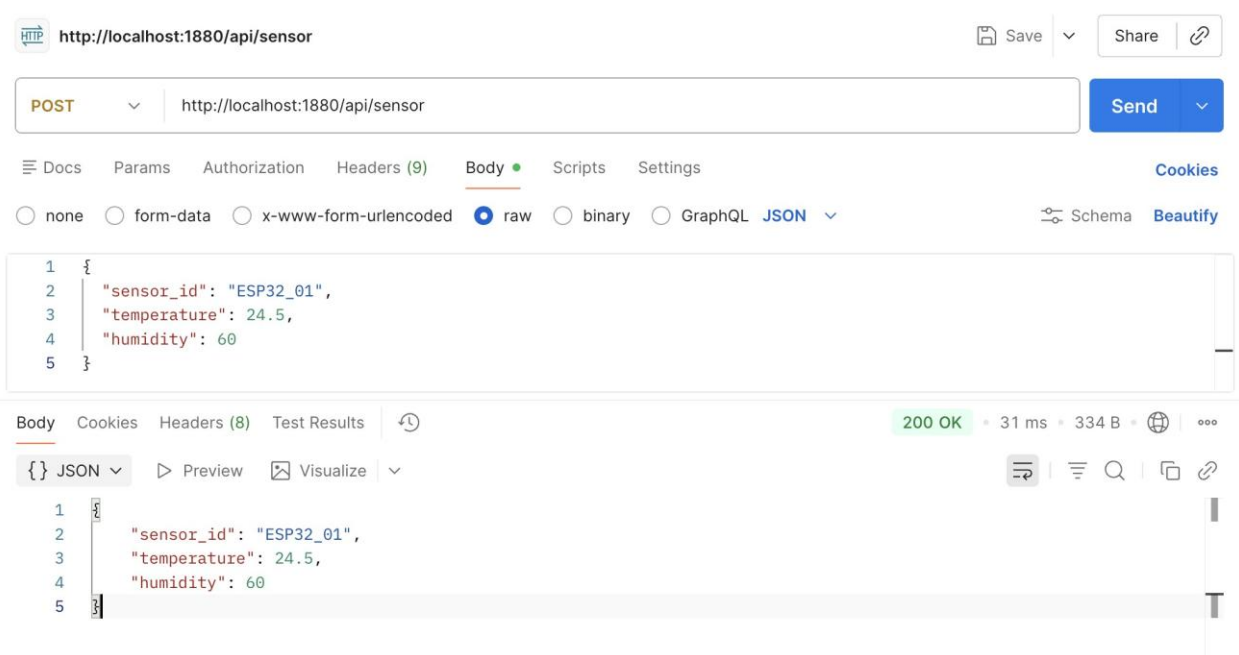
1. **Launch Postman:** Open the Postman application and click **New > HTTP Request**.
2. **Configure the Request Method & URL:** * Change the dropdown from GET to POST.
 - o Enter your endpoint URL: `http://localhost:1880/api/sensor`

(Note: To test from another machine on the same network, replace 'localhost' with your computer's IPv4 address).

3. **Construct the JSON Payload:**
 - o Click the **Body** tab underneath the URL bar.
 - o Select the **raw** radio button.
 - o Change the text format dropdown from 'Text' to **JSON**.
 - o Enter a simulated sensor payload:

```
JSON
{
  "sensor_id": "ESP32_01",
  "temperature": 24.5,
  "humidity": 60
}
```

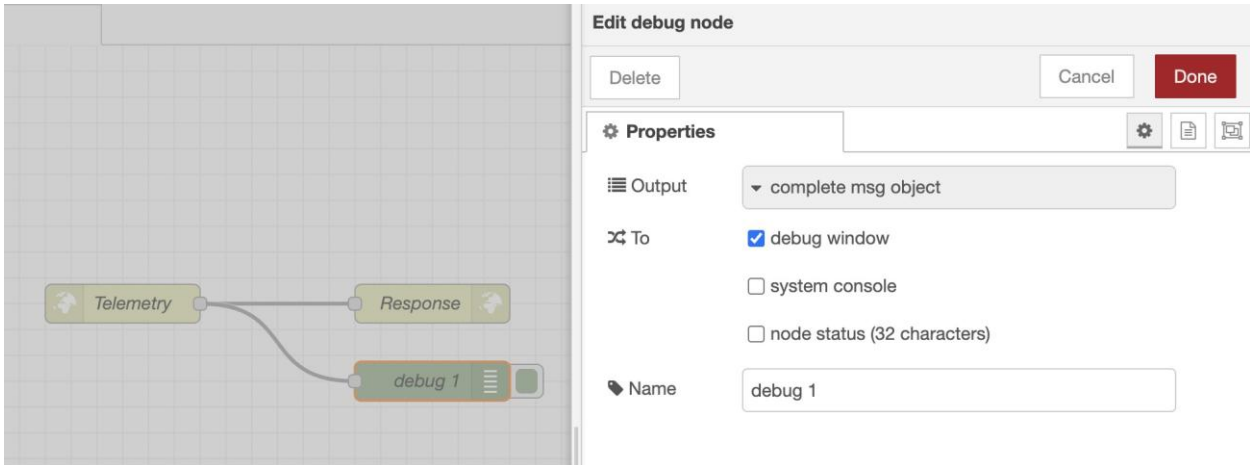
4. **Send and Verify:** Click the blue **Send** button.
 - o **In Postman:** You should see a Status: 200 OK at the bottom, confirming the server received the request.
 - o **In Node-RED:** Look at your Debug sidebar. You should see the complete JSON object you just sent successfully logged in the console!



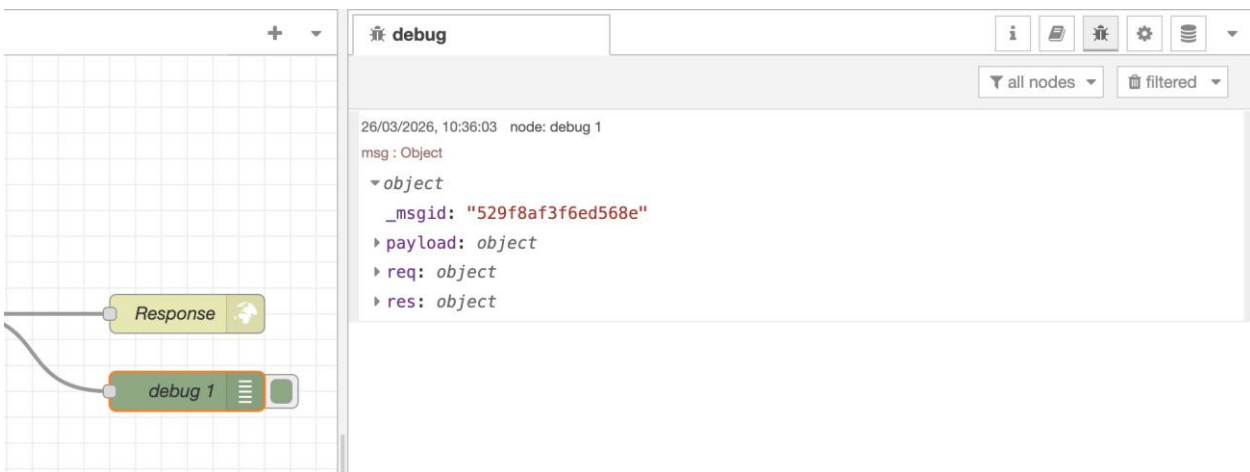
Step 4: Debugging the Node-RED Flow

In distributed systems, silent failures are dangerous. We must set up a console log to inspect the incoming HTTP payloads.

1. **Add a Debug Node:** Drag a green `debug` node onto the workspace.
2. **Wire for Inspection:** Wire the output of your http in node to the input of the debug node (it will share the connection with the response node).
3. **Configure Debug Output:** Double-click the debug node. Change the output from `msg.payload` to **complete msg object**. This allows you to inspect the HTTP headers, the method, and the raw JSON payload.



4. **Open the Console:** Click the small "Bug" icon on the far right sidebar to open the Debug Messages panel.
5. **Deploy:** Click **Deploy** again to save the debugging logic.
6. Send the request again from postman, and you should be able to see the debug message.



Discussion

To verify your understanding of RESTful architectures, complete the following challenges and answer the questions below.

A: HTTP Method

1. In your Node-RED workspace, duplicate your `http in` node and change the Method from POST to GET.
2. Add a `function` node in between `http in`, and `http response`. Then in the **On Message** tab return any JSON payload to the msg. Click **Deploy**.
3. In Postman, change your request method to `GET` and attempt to send the request.

What happened to your JSON payload when you switched to a GET request? Did Node-RED successfully log the temperature data in the debug window? Explain the fundamental architectural difference between how a POST request and a GET request handle data payloads.

The Payload: When you switch to GET , your JSON data disappears. Web browsers and API tools like Postman strip the "body" out of GET requests by default.

The Debug Log: Because the data is stripped, Node-RED will not log the temperature data. The payload will arrive empty.

POST (Sending Data): Carries data hidden inside the Request Body. It is designed for posting or saving data (like sending sensor logs) and has no size limits.

GET (Asking for Data): Designed strictly to retrieve data. It cannot hold a data body; if it must send information, it has to stick it directly onto the end of the URL text (eg, `?temp=24.5`).

B: Local Area Network (LAN) Edge Connectivity

Your Docker container is currently bound to your physical computer's port 1880. Let's prove that this acts as a true local edge server.

1. Find your computer's local IPv4 address (e.g., `192.168.1.X`) using `ipconfig` (Windows) or `ifconfig` (macOS/Linux) in your terminal.
2. Ensure your smartphone is connected to the exact same Wi-Fi network as your computer.
3. Open the web browser on your smartphone and type:
`http://[YOUR_IPV4_ADDRESS]:1880/api/sensor`

Did your phone successfully trigger the Node-RED flow? Explain why you are not able to send a request to the POST method from your phone?

= Yes. Because your Docker settings connect the port 1880 of the container directly to your physical computer's network card, any device on the same Wi-Fi network can reach your server using your computer's IP address

Why can't you send a POST request from your phone's browser?

= When you type a web link into a mobile browser's address bar, the phone can only send a GET request. Because you coded your Node-RED endpoint to strictly listen for POST requests, it ignores the phone's GET request.

C: Deep Packet Inspection via Debugging

Analysing both the debug message in NodeRed for both activities **A** and **B** above, expand the entire `msg` object by clicking the small arrows next to the logged message.

- Look deeply at the expanded `msg.req` (request) object. Identify and list the following components involved in your HTTP protocol transmission:
 - a) What is the `content-type` listed under the `headers`?
 - b) What is the IP address of the client that sent the request (look for `remoteAddress`)?
 - c) Compared the differences between **A** and **B**

a) Content Type: -**Activity A (Postman)**: application/json.
-**Activity B (Phone)**: Blank/None, or text/html

b) Client IP Address (`remoteAddress`):

Activity A: 127.0.0.1 (or ::1). This means "Localhost" the request came from the exact same computer running the server.

Activity B: The actual local IP address of your mobile phone on the Wi-Fi network (eg, 192.168.1.15).

c)

Features	Activity A (Postman)	Activity B (Smartphone)
HTTP Method	POST	GET
Source IP	127.0.0.1(Your PC)	192.168.1.X(Your Phone)
Data Payload	Yes (JSON object is intact)	No (Arrives empty)
App Type	API Testing Tool (Postman)	Standard Web Browser